Software Engineering Group Assignment 1: Smart Ticketing System

- **Reference Document:** Please refer to the file named "LabGroupAssignment1.pdf" for initial context and submission guidelines.
- **Instructor:** P.T.K.Trung (phamthaikytrung@tdtu.edu.vn) | Copyright @ Sep2026-SE
- **Deadline: 31 Oct 2026. Weight:** 20% of QT2 (Plus 1 extra bonus point towards the Final Project for exceptional work)
- **Format:** Group Assignment (3-5 students)
- **Total Marks:** 100%

1. Business Context & Problem Statement

The following statements represent the currently *omitted and ambiguous* requirements for a **Ticket Vendor Machine (TVM)**, which is part of a broader Smart Ticketing System.

An automated ticket-issuing system sells public transportation tickets (Bus, MRT, etc.). Users select their destination and their preferred mode of payment (Credit Card, QR Code linked to a banking system, or Digital Wallet).

If paying by credit card, the ticket vendor machine charges the user's account and issues a paper ticket with a printed barcode. The physical interaction is as follows: When the passenger presses the "Start" button, a menu of potential destinations is displayed, prompting the user to make a selection. Once a destination is selected, the user is asked to insert/tap their credit card. Upon validation of the credit transaction, the ticket is issued.

If paying via a digital wallet, the process is similar, but instead of inserting a card, the TVM will generate and display a QR Code on the screen after the passenger selects their route. The user scans this with their mobile phone to complete the payment.

2. Your Tasks (100% Total)

Assume the role of a Software Development Specialist. Your team is required to complete the following phases:

Phase 1: Requirements Engineering (20%)

1. **Clarification Q&A (Excel) [5%]:** Identify the ambiguities in the provided problem statement. Formulate a list of clarifying questions you would ask the stakeholders, and provide logical, assumed answers to base your design on.
2. **Requirements Specification (Word/Excel) [15%]:** Write a formal set of Functional, Non-Functional, and Domain requirements for the TVM.
    - *Critical Focus:* Pay special attention to defining exact expectations for **reliability** and **response time**.

Phase 2: UML Modeling & System Design (35%)

3. **Use Case Modeling [10%]:** Develop a Use Case Diagram for the TVM. Include detailed Use Case Descriptions for each identified use case.
4. **Activity Diagrams [10%]:**
    - Create an Activity Diagram illustrating the step-by-step process of a passenger buying a ticket from the TVM (similar to an ATM flow).
    - Create a separate Activity Diagram showing the communication and integration between your system and external payment gateways (e.g., Momo, VNPay, ZaloPay).
5. **Detailed System Modeling (Main Use Case: "Buy a Ticket") [15%]:**
    - **Sequence Diagram:** Show the object interactions over time.
    - **State Chart Diagram:** Show the states the TVM goes through during the transaction.
    - **Class Diagram:** Define the core domain model (classes, attributes, methods, relationships).

Phase 3: UI/UX & Architecture (15%)

6. **UI/UX Design [5%]:** Design wireframes or mockups for the TVM screens using Balsamiq, Figma, or a similar tool.
7. **Architectural Design [10%]:** Develop an Architecture Design diagram (System-in-a-box or MVC model) and a Deployment Diagram showing how the TVM hardware, servers, and external APIs connect.

Phase 4: Implementation, Tooling & Demo (30%)

8. **Visual Studio Engineering (Model-to-Code) [10%]:** Utilize **Visual Studio** to demonstrate bidirectional engineering.
    - Perform **Forward Engineering** (Model to Code) to generate initial C# code skeletons from your class diagrams.
    - Perform **Backward Engineering** (Code to Model) to reflect any manual code changes back into your UML architecture.
    - Generate a **Code Map** in Visual Studio to visually present the relationships and architecture of your compiled code.
9. **GitHub & AI Tooling (Copilot) [10%]:**

- Source Control: Host your project on **GitHub**. Your repository must show a committed history demonstrating active collaboration among group members.
- AI Assistance: Use **GitHub Copilot** (or similar AI tools) to assist your development. In your report, you must include a section documenting the prompts you used, how Copilot assisted in generating boilerplate or complex logic, and how you validated the AI-generated code.

10. **Demo Application [10%]:** Develop a functional prototype demo of the main use case using Visual Programming **C#** and **MSSQL**. This must include an input form and an output report (or success screen).

3. Submission Guidelines

Ensure all final artifacts are uploaded to a shared GitHub repository. Your final submission to the portal must include:

1. **[YourID]_[YourStudentName].docx**: A comprehensive report containing all your diagrams, requirements, AI prompt documentation, and screenshots explaining your work products. Ensure the content of "LabGroupAssignment1.pdf" is addressed.
2. **readme.txt**: Must contain the link to your public/accessible GitHub repository and any other necessary links (e.g., Figma link, database script instructions).
3. **% Contribution of each member**

Enjoy it ☺

Grading Rubric (Student Reference)

Category	Excellent (Full Marks)	Developing (Partial Marks)	Needs Improvement (Low/No Marks)
Requirements (20%)	Thorough Q&A uncovering hidden complexities. Strict, measurable non-functional limits defined.	Basic Q&A. NFRs are mentioned but lack measurable metrics (e.g., "fast response").	Missing Q&A or severely lacking requirement definitions.
UML & Design (35%)	Diagrams are syntactically perfect, logically sound, and perfectly aligned with each other.	Diagrams are mostly correct but have minor UML syntax errors or slight logical gaps.	Missing major diagrams or severe misunderstandings of UML principles.
UI & Arch (15%)	Modern, intuitive UI. Architecture is highly logical, modular (MVC), and suitable for hardware.	Basic UI. Architecture diagram is present but overly simplistic or lacks API context.	No wireframes or highly flawed architectural logic.
Tooling & Process (20%)	Exceptional use of VS Code Maps, Forward/Backward Engineering. Excellent documentation of Copilot prompts. Rich GitHub history.	Tools used basically. Missing some engineering steps. Copilot usage stated but poorly documented. Minimal commits.	Failed to use Visual Studio engineering tools. No GitHub history. No Copilot documentation.
Implementation (10%)	C#/MSSQL demo runs perfectly, capturing the core ticket buying flow smoothly.	Demo runs but has bugs or only partially completes the ticket buying flow.	Demo crashes, fail to connect to DB, or not submitted.